COURSE: CS - 4173 - COMPUTER SECURITY

SEMESTER: SPRING 2026

INSTRUCTOR: DR. SHANGQING ZHAO

GROUP MEMBERS: RUBY MORALES, TRINH NHAN, IVAN FRAIRE

Introduction

Secure communication is an important part of modern computing because data sent across a network can be intercepted or exposed if it is not properly protected. For this project, a secure instant point-to-point messaging tool was designed and implemented to allow two users to exchange messages privately over a network. The tool follows a client/server model and provides encrypted communication between both users while also demonstrating important computer security concepts in a practical way.

The messaging application allows both users to enter the same shared password, derive a secure encryption key from that password, and use that key to encrypt and decrypt messages through a graphical user interface. The assignment required that the password not be used directly as the encryption key, that each transmitted message be encrypted with a key length of at least 56 bits, that repeated messages produce different ciphertext when possible, that the GUI display ciphertext and plaintext, and that the system include a mechanism for periodic key updates. We met these requirements by making our design use AES-128 for encryption, PBKDF2 for key derivation, TCP sockets for communication, a random IV for every message, and a Tkinter-based GUI for sending and receiving messages. The system also updates the derived key after every 20 messages, helping reduce the amount of communication protected under a single key.

Cipher Choice

AES (Advanced Encryption Standard) encryption was chosen for the design because it’s secure and widely trusted. It’s the global standard for symmetric encryption and is regularly used in real systems, as well as supporting multiple key sizes. In the design, AES-128 is used, which provides a 128-bit key, making it stronger and more secure than the 56-bit key requirement. A 128-bit key provides the design with high security and makes brute-force attacks more difficult to deploy.

The design uses CBC (Cipher Block Chaining) mode because it's simple to implement and requires a random IV for each message. Using a new IV in each message helps prevent identical plaintext from producing identical ciphertext, which reduces pattern leaks. Since the design generates a new random IV for every message, CBC provides semantic security and keeps data from being hacked.

Additionally, AES is efficient in both hardware and software, which is why it’s so widely used in many real-world systems such as VPNs and secure messaging applications. It’s been publicly analyzed for many years with no practical attacks, making it more reliable.

Key Derivation

The PBKDF2 function is used for key derivation because using raw passwords directly is weak and predictable. Human-chosen passwords are easy to attack, which becomes a vulnerability for the entire system. This is why PBKDF2 is used, because it transforms the user's password into a strong 128-bit AES key instead of using the password itself. The function provides security by applying a cryptographic hash function for thousands of iterations, making brute-force attacks much slower and still strengthening the password. This design slows attackers while remaining fast enough for legitimate users.

A salt is a random value added to the password before hashing. It ensures that two users with the same password will still generate different keys, preventing rainbow-table attacks. Rainbow-table attacks are a method attackers use to crack passwords quickly in systems that do not use salts. The code generates a random salt and uses it in PBKDF2, which produces a unique AES key each session, even if the same password is reused.

PBKDF2 is deterministic, meaning that if both sides use the same password, salt, iteration count, and key length, they will derive the same AES-128 key. Since the client and server share the same parameters, both users will always generate the same key. This allows them to encrypt and decrypt each other's messages securely and privately.

Padding Scheme

Since AES encrypts data in fixed-size 16-byte blocks, and chat messages are not always multiples of 16 bytes, the encryption process requires padding to fill the final block to the correct size. For the padding scheme, PKCS7 padding is used because the library in my design automatically applies the padding during encryption and removes it during decryption. PKCS7 is widely used for block ciphers, is secure, and compatible with AES-CBC. It works by adding bytes to indicate how many padding bytes were added, making the padding easier and safer to remove after the decryption.

Network Design

The messaging tool we built relies on a straightforward networking design where Bob acts as the server, and Alice connects to him as a client. This setup serves to be like a real point-to-point communication scenario, where one user can host the conversation, and the other joins it. In our implementation, Bob starts by launching the gui_server.py, which binds to port 5003 and then waits for an incoming connection. Once the server is up and running, Alice starts gui_client.py, enters Bob's IP address, and

establishes a TCP connection to his machine. From that moment on, all encrypted messages flow directly between Alice and Bob through this socket connection.

The server, in simple it listens for connections and maintains the socket once Alice joins. After the connection is established, both sides immediately start a background thread dedicated to receiving incoming messages. This design keeps the GUI responsive because receiving data never blocks the user from typing or sending their own messages. Without the receiving thread, the interface would just freeze every time the program requested new data. Instead, the GUI remains smooth, and messages appear in real time as they arrive between Alice and Bob. This threading model is exactly what allows Alice and Bob to chat naturally, even though every message is being encrypted, decrypted, and processed behind the scenes.

Using raw TCP sockets ended up being the best fit for us in this project because they gave us a persistent and reliable communication channel without unnecessary overhead. He guarantees that messages arrive in order and without loss or getting mixed up, which was really important for a secure messaging tool. There's no polling, no repeated HTTP requests, and no delays. It's a continuous stream of encrypted data flowing between Alice and Bob. The socket layer essentially became the backbone of our system, with all the cryptographic logic sitting on top of it. This separation keeps the network simple and security strong.

On top of this networking structure, the system implements two major security features. That both the content of the messages and the long-term privacy of a conversation remain private. A random IV for every message and a periodic key update mechanism did so. These features go beyond basic encryption and help address real-world concerns like pattern leakage and long-term key exposure.

The first major feature is the random initialization vector. For every message Alice and Bob sent in AES CBC mode. The IV plays an important role in making sure that even if two plain-text messages are exactly the same, the ciphertext will look completely different. Without a random IV, repeated messages would be produced. Repeated ciphertext blocks would then immediately expose patterns to anyone observing the network traffic. For example, if Alice sends hello multiple times, an attacker can recognize that repetition just by comparing the ciphertext, even without having to decrypt anything, which alone leaks information about the structure of the conversation.

By generating a fresh random IV for every message, our system prevents this kind of pattern analysis. Each message becomes a unique encryption event, independent from all others. Even if Alice and Bob

send the same phrase over and over again 10 times, the ciphertext will never repeat. This will help protect against attackers because an attacker can't correlate messages or guess which ones might be related. The program handles IV generation automatically, so neither Alice nor Bob has to actively think about it. The security is built into the encryption process itself. This is the same principle used in real secure messaging apps, where IV reuse is considered a serious vulnerability.

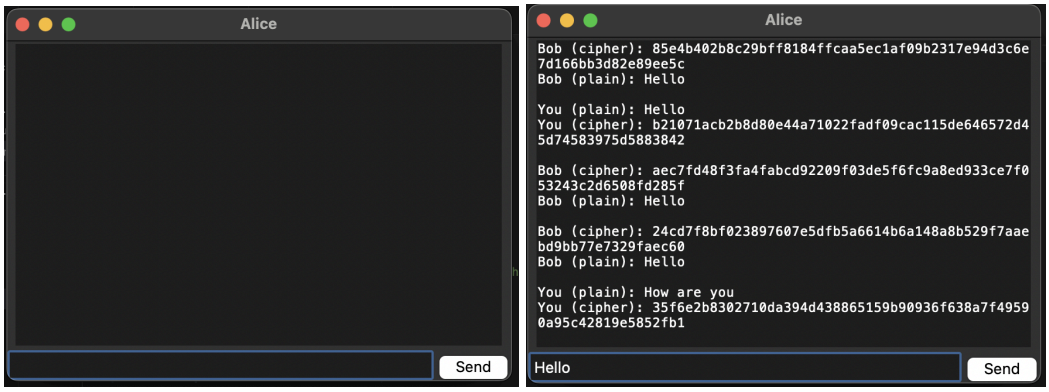
The second major security feature is the periodic key update mechanism, which strengthens the system over longer conversations. This meant that instead of relying on a single static key for the entire session, the system automatically updates the encryption key every 20 messages. It does this by incrementing a rotationCount value and feeding that into PBKDF2 along with the original password. Because PBKDF2 is deterministic, both Alice and Bob will always derive the same new key as long as they share the same password on the same rotationCount. This design gives the system a lightweight form of forward secrecy; even if an attacker were somehow able to get an older key, they would only be able to decrypt the messages from that specific rotation. All future ones would be protected by a new key that an attacker doesn't have.

This key update mechanism also limits the amount of data encrypted under any single key, which reduces the risk of long-term cryptanalysis. In real-world systems, the longer a key is used, the more opportunities an attacker has to try and break it or gather enough ciphertext to attempt analysis. By refreshing our keys regularly, the system minimizes risk and ensures that even long conversations stay secure. Our program then prints a “[KEY UPDATE”] message in the terminal whenever the key rotates, giving Alice and Bob transparency without requiring them to manage anything manually. The update happens instantly and silently in the background, allowing the conversation to keep going on without any interruption.

Together, the networking design and security features create a messaging tool that is both functional and secure. The socket-based architecture ensures real-time communication between Alice and Bob, while the random IV and periodic key updates protect the confidentiality and integrity of messages. These features reflect real principles used in modern secure messaging applications, where encryption is not just about scrambling, but also preserving patterns and limiting exposures, and ensuring that past messages remain safe even if future keys are compromised by combining all of these elements. The project shows how networking and cryptography work together to create a privacy preserving communication system.

GUI Demonstration:

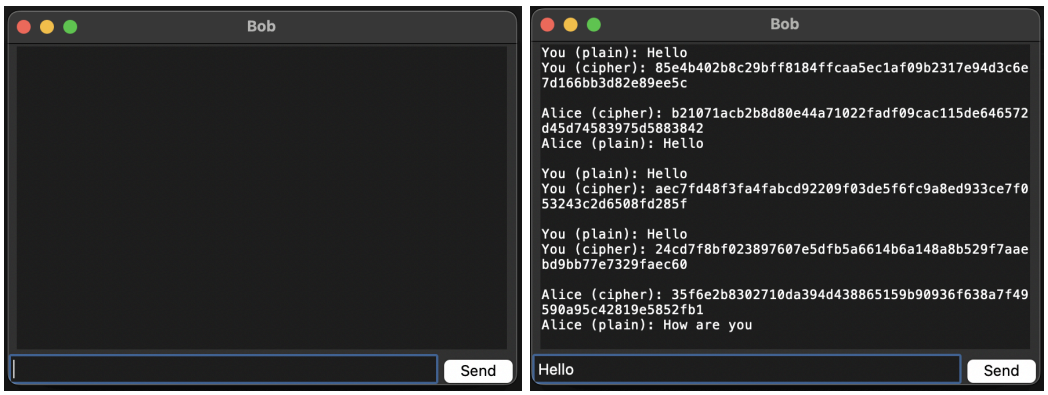
Client GUI:




```
ivanfraire@Ivans-MacBook-Pro-3 Instant-Point-to-Point-Messaging-Tool % cd secure_chat
ivanfraire@Ivans-MacBook-Pro-3 secure_chat % python3 gui_client.py
Enter shared password: Oklahoma
```

These screenshots show the client-side interface used by Alice to send and receive messages. It demonstrates that the system includes a working graphical user interface with a chat display area, message input field, and send button.

Server GUI:




```
ivanfraire@Ivans-MacBook-Pro-3 Instant-Point-to-Point-Messaging-Tool % cd secure_chat
ivanfraire@Ivans-MacBook-Pro-3 secure_chat % python3 gui_server.py
Waiting for connection...
Connected to: ('127.0.0.1', 62475)
Enter shared password: Oklahoma
```

These screenshots show the server-side interface used by Bob after accepting a connection from the client. It demonstrates that both users can interact through a graphical interface and that the system supports secure communication between connected users.

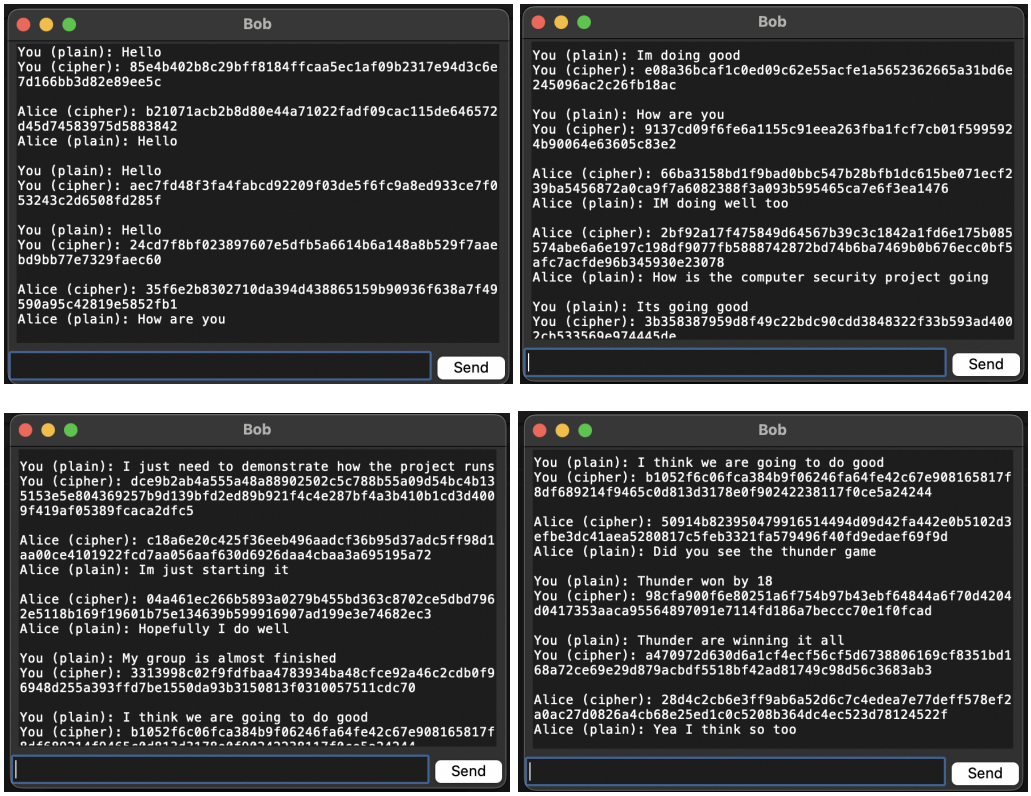
Conclusion:

Overall, this project brought together the core ideas of computer security and helped us apply them in a practical hands-on way by building a point of point messaging tool from scratch we were able to see how concepts like key derivation block cipher modes, padding, secure networking actually work when they're implemented in real code our design met all of the assignment requirements we used PBKDF2 to derive a strong AES 128 key from a shared password encrypted every message with a fresh random IV and updated it after every 20 messages to make sure that we didn't cause long term exposure. The network layer built on a simple TCP socket connection between Allison and Bob allow us to demonstrate secure communication in real time while keeping the system easy to understand.

Putting these pieces together showed how each part of security pipeline depends on the others the random IV prevents pattern leak the periodic key updates reduce the amount of data protected under a single key and the socket based design made sure that encrypted messages move reliably between both of the users, Allison and Bob. Working on all these details helped to understand how encryption works and also why secure systems are designed and made in the way that they are. Overall the project showed all the concepts of our class, especially cryptography, networking and software design, all coming together to create a working and secure communication tool

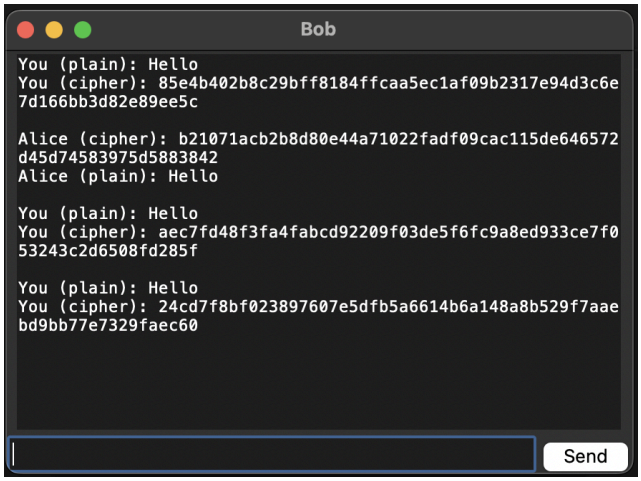
Sent and Received Plaintext and Ciphertext:

Conversation (Bob's POV):



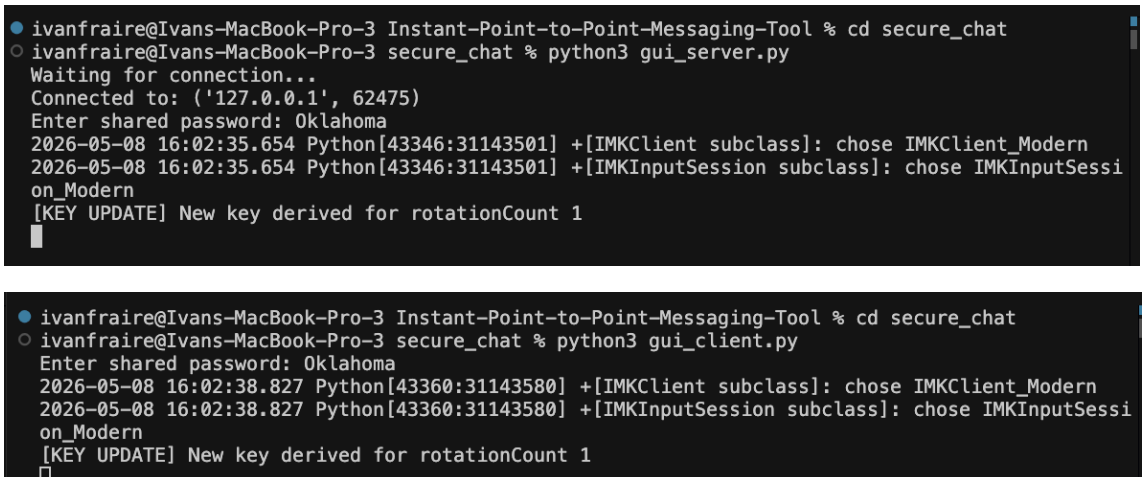
The screenshots show that the messaging tool displays both plaintext and ciphertext during communication. When a user sends a message, the GUI shows the original plaintext along with the encrypted ciphertext. When a message is received, the GUI displays the received ciphertext and the correctly decrypted plaintext, demonstrating that the system successfully performs both encryption and decryption.

Repeated message with different ciphertext:



This screenshot shows the same plaintext message, “Hello,” being sent multiple times while producing different ciphertext values each time. This happens because the system generates a fresh random IV for every message, which prevents identical plaintext from resulting in identical ciphertext and helps reduce visible message patterns.

Key Update Terminal Output:



These terminal outputs show that both the client and server automatically derive a new key after reaching the update threshold (20 messages). This demonstrates that the design includes a periodic key update mechanism, helping reduce the amount of communication protected under one derived key.

Extra Credit

Question1:

The security of the messaging system can be improved by using a layered encryption design instead of relying on a single cipher. In this design, the plaintext message would first be encrypted using AES-128-CBC with one derived key, and then the resulting ciphertext would be encrypted again using a second cipher, such as ChaCha20, with a different independently derived key. This approach increases security by adding two separate encryption layers based on different cryptographic designs. Even if one layer were weakened, the second layer would still protect the message. Both AES and ChaCha20 are widely used and optimized standard ciphers, so the design would stay practical for a messaging application. It would also provide stronger protection than using only one encryption method.

Question2:

For this bonus feature, the project was modified so that Alice and Bob do not need to start with the same shared password. Instead, both users use RSA public/private key pairs to authenticate each other during the connection process. After both sides are verified, a random AES session key is securely shared and used to encrypt the chat messages. This allows the system to establish secure communication without depending on a pre-shared password.